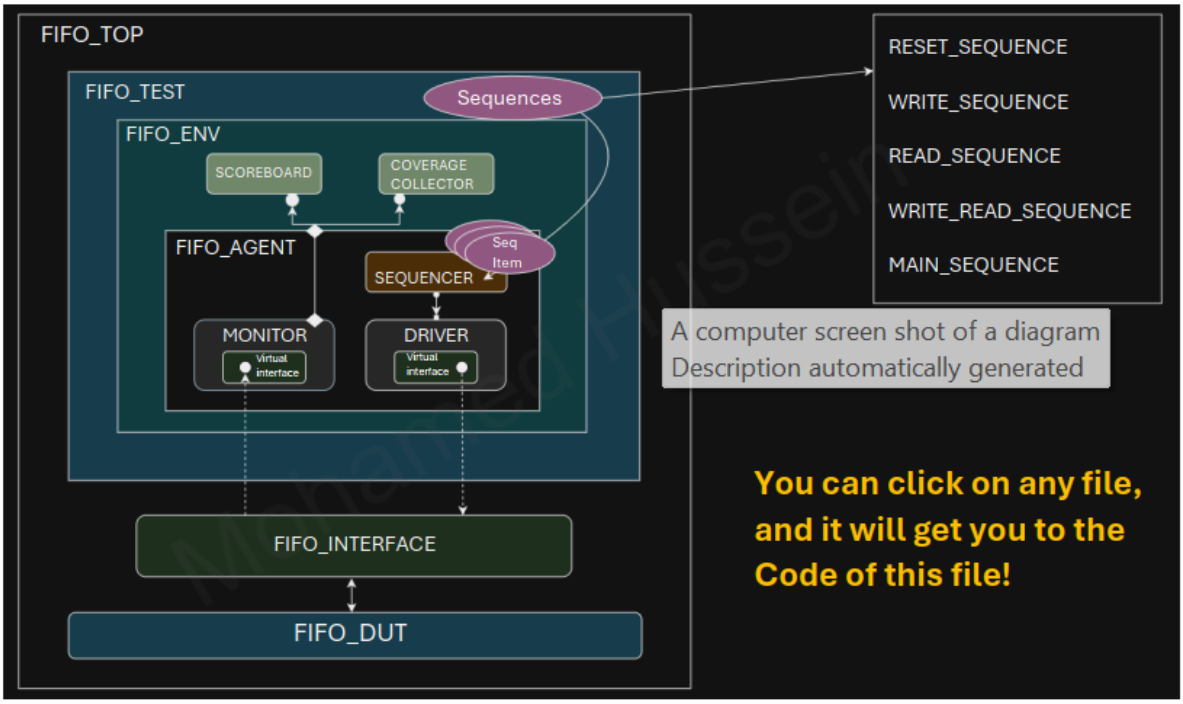
FIFO UVM Structure:



How it Works?

1. Top Module (FIFO_TOP.sv):

This is the highest level of your verification environment. It instantiates the design under test (DUT) and connects the DUT to the UVM testbench. The FIFO_TOP.sv module also instantiates the FIFO_IF (interface) and links it to the DUT, ensuring signals between the testbench and the DUT are connected correctly.

2. Interface (FIFO_IF.sv):

The FIFO_IF.sv defines the signals that connect to the FIFO DUT. This includes signals like clock, reset, write enable, read enable, data inputs, and outputs. The interface provides a structured mechanism to communicate with the DUT by grouping related signals together and making them accessible to the driver, monitor, and scoreboard.

3. UVM Testbench Flow:

The UVM testbench drives the verification process by generating stimulus, driving it into DUT, monitoring the outputs, and analyzing the results.

3.1 Configuration (FIFO_CONFIG.sv)

The configuration object holds parameters like FIFO depth, data width, and other operational settings. This configuration is passed to various components in the testbench to ensure a consistent environment for verification.

3.2 Sequences and Sequence Items

- **FIFO_SEQUENCE_ITEM.sv:**

This file defines the transaction object (sequence item) that carries the data

and control signals for write and read operations to/from the FIFO. It

abstracts the data transfer operations.

- **FIFO_ SEQUENCE**

This file define different sequences, which are collections of transactions.Each sequence class has a specific operation: resetting the FIFO, writing data,reading data, performing combined write-read operations, and orchestrating the overall verification flow (in the main sequence). The main sequencecontrols the order of operations and ensures different sequences areexecuted as part of the verification plan.

3.3 Sequencer (FIFO_SEQUENCER.sv)

The sequencer coordinates the execution of sequences. It sends the transaction (sequence item) to the driver, which will then apply it to the DUT. In your UVM testbench, the FIFO_SEQUENCER manages the flow of read and write sequences and other operations, ensuring the correct stimulus is generated.

3.4 Driver (FIFO_DRIVER.sv)

The driver takes the transaction objects from the sequencer and translates them into pin-level activity on the DUT using the interface. It drives the stimulus into the DUT through the FIFO_IF, such as asserting write or read enable signals, and providing the data input to the FIFO.

3.5 Monitor (FIFO_MONITOR.sv)

The monitor observes the DUT's signals through the interface. It passively captures the inputs and outputs of the FIFO for analysis without modifying the DUT's behavior. The monitor collects relevant data and forwards it to other components like the coverage collector and scoreboard.

4. Analysis Components

4.1 Scoreboard (FIFO_SCOREBOARD.sv)

The scoreboard is responsible for checking the functional correctness of the DUT. It compares the expected results (predicted by a reference model or logic) with the actual outputs observed by the monitor. In your project, the FIFO_SCOREBOARD will verify that

the data written into the FIFO is correctly read out and that any underflow or overflow conditions are handled properly.

4.2 Coverage Collector (FIFO_COVERAGE_COLLECTOR.sv)

The coverage collector gathers coverage information, tracking how thoroughly the DUT has been tested. It looks at functional coverage metrics like whether all possible write-read combinations, FIFO full/empty conditions, and other critical scenarios have been exercised. This component provides insight into how much of the design has been verified.

5. Environment (FIFO_ENV.sv)

The environment is the container for all the UVM components, such as agents, scoreboard, and coverage collector. The FIFO_ENV instantiates the agents that control the driver, monitor, and sequencer and ensures that these components interact correctly. It also connects the environment's agents to the scoreboard and coverage collector.

• FIFO_AGENT.sv:

The agent encapsulates the driver, monitor, and sequencer for the FIFO. It

coordinates their activities to ensure the interface is driven correctly and monitored

consistently. The agent simplifies managing these components by providing a

unified entity to control them.

6. Assertions (FIFO_SVA.sv)

The assertions module (FIFO_SVA.sv) defines SystemVerilog assertions (SVA) to check critical design properties. For example, assertions may ensure that the FIFO never overflows or underflows and that data integrity is maintained. These are runtime checks that trigger if the DUT violates any of the expected behaviors.

7. Test (FIFO_TEST.sv)

The test file (FIFO_TEST.sv) is where the overall test strategy is defined. This file extends the UVM test class and initiates the execution of the sequences. It configures the environment, runs the main sequence, and ensures the test runs as expected. You may have multiple tests targeting different aspects of the FIFO (e.g., stress tests, boundary condition tests,

Bugs Report:

1. Underflow signal implementation issue

- underflow is implemented as a combinational signal instead of a sequential one.

- Modification : made it following clock edge in the read always block

2. Missing conditions

- If both read and write enables are high simultaneously:

    o When the FIFO is empty, only writing will take place.

    o When the FIFO is full, only reading will take place.

3. Reset Behavior error

- Bug: there are some signals which their output haven’t been specified when reset signal takes place

```systemverilog
import uvm_pkg::*;
import FIFO_test_pkg::*;
`include "uvm_macros.svh"

module FIFO_top();
    logic clk;

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    FIFO_if fifoif(clk);

    FIFO #(.FIFO_WIDTH(fifoif.FIFO_WIDTH),.FIFO_DEPTH(fifoif.FIFO_DEPTH))
        dut (
            fifoif.data_in,
            fifoif.wr_en,
            fifoif.rd_en,
            fifoif.clk,
            fifoif.rst_n,
            fifoif.full,
            fifoif.empty,
            fifoif.almostfull,
            fifoif.almostempty,
            fifoif.wr_ack,
            fifoif.overflow,
            fifoif.underflow,
            fifoif.data_out
        );

    initial begin
        uvm_config_db #(virtual FIFO_if)::set(null,"uvm_test_top","FIFO_IF",fifoif);
        run_test("FIFO_test");
    end

endmodule
```

FIFO_if.sv

```systemverilog
interface FIFO_if(input logic clk);
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    logic [FIFO_WIDTH-1:0] data_in;
    logic rst_n, wr_en, rd_en;
    logic [FIFO_WIDTH-1:0] data_out;
    logic wr_ack, overflow;
    logic full, empty, almostfull, almostempty, underflow;

endinterface //FIFO_if
```

FIFO_test.sv

```
package FIFO_test_pkg;
import uvm_pkg::*;
import FIFO_config_pkg::*;
import FIFO_env_pkg::*;
import FIFO_sequence_pkg::*;

`include "uvm_macros.svh"

    class FIFO_test extends uvm_test;
        `uvm_component_utils(FIFO_test);

        // add environment class
        FIFO_env env;
        // add sequence class
        FIFO_reset_sequence reset_seq;
        FIFO_write_sequence wr_seq;
        FIFO_read_sequence rd_seq;
        FIFO_rd_wr_sequence rd_wr_seq;
        //FIFO_main_sequence main_seq;
        virtual FIFO_if fifoif;
        FIFO_config FIFO_cfg;

        function new(string name = "FIFO_test", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            env = FIFO_env::type_id::create("env",this);
            FIFO_cfg = FIFO_config::type_id::create("FIFO_cfg");
            reset_seq = FIFO_reset_sequence::type_id::create("reset_seq",this);
            wr_seq = FIFO_write_sequence::type_id::create("wr_seq",this);
            rd_seq = FIFO_read_sequence::type_id::create("rd_seq",this);
            rd_wr_seq = FIFO_rd_wr_sequence::type_id::create("rd_wr_seq",this);

            if(!uvm_config_db#(virtual FIFO_if)::get(this,"","FIFO_IF",FIFO_cfg.fifo_vif))
                `uvm_fatal("build phase","Test - Unable to get the virtual interface");

            uvm_config_db#(FIFO_config)::set(this,"*","CFG", FIFO_cfg);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);

            phase.raise_objection(this);

            `uvm_info("run_phase","Reset asserted", UVM_LOW)
            reset_seq.start(env.agt.sqr);
            `uvm_info("run_phase","Reset dasserted", UVM_LOW)

            `uvm_info("run_phase","Stimulus generation started", UVM_LOW)
            wr_seq.start(env.agt.sqr);
            rd_seq.start(env.agt.sqr);
            rd_wr_seq.start(env.agt.sqr);
            `uvm_info("run_phase","Stimulus generation ended", UVM_LOW)

            #10;
            phase.drop_objection(this);
        endtask
    endclass
endpackage
```

FIFO_env.sv

```
package FIFO_env_pkg;
import uvm_pkg::*;
import FIFO_config_pkg::*;
import FIFO_agent_pkg::*;
import FIFO_scoreboard_pkg::*;
import FIFO_coverage_pkg::*;
`include "uvm_macros.svh"

    class FIFO_env extends uvm_env;
        `uvm_component_utils(FIFO_env);

        // add agent class
        FIFO_agent agt;
        // add scoreboard class
        FIFO_scoreboard sb;
        // add coverage class
        FIFO_coverage cv;

        function new(string name = "FIFO_env", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            agt = FIFO_agent::type_id::create("agt",this);
            sb = FIFO_scoreboard::type_id::create("sb",this);
            cv = FIFO_coverage::type_id::create("cv",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
           agt.agt_ap.connect(sb.sb_export);
           agt.agt_ap.connect(cv.cv_export);
        endfunction

    endclass
endpackage
```

FIFO_config.sv

```
1   package FIFO_config_pkg;
2   import uvm_pkg::*;
3   `include "uvm_macros.svh"
4
5       class FIFO_config extends uvm_object;
6           `uvm_object_utils(FIFO_config);
7
8           virtual FIFO_if fifo_vif;
9
10          function new(string name = "FIFO_config");
11              super.new(name);
12          endfunction
13
14      endclass
15
16  endpackage
```

```
1   package FIFO_sequence_pkg;
2   import uvm_pkg::*;
3   import FIFO_config_pkg::*;
4   import FIFO_sequence_item_pkg::*;
5   `include "uvm_macros.svh"
6
7       class FIFO_reset_sequence extends uvm_sequence #(FIFO_sequence_item);
8           `uvm_object_utils(FIFO_reset_sequence)
9
0           FIFO_sequence_item seq_item;
1
2           function new(string name = "FIFO_reset_sequence");
3               super.new(name);
4           endfunction
5
6           task body();
7               seq_item = FIFO_sequence_item::type_id::create("seq_item");
8               start_item(seq_item);
9               seq_item.rst_n= 0;
0               seq_item.data_in = 0;
1               seq_item.wr_en= 0;
2               seq_item.rd_en= 0;
3               finish_item(seq_item);
4           endtask
5       endclass
6
```

```
26
27      class FIFO_write_sequence extends uvm_sequence #(FIFO_sequence_item);
28          `uvm_object_utils(FIFO_write_sequence)
29
30          FIFO_sequence_item seq_item;
31
32          function new(string name = "FIFO_write_sequence");
33              super.new(name);
34          endfunction
35
36          task body();
37              seq_item = FIFO_sequence_item::type_id::create("seq_item");
38
39              repeat(10) begin
40                  start_item(seq_item);
41                  seq_item.rst_n= 1;
42                  seq_item.data_in = $urandom_range(0, 255);
43                  seq_item.wr_en= 1;
44                  seq_item.rd_en= 0;
45                  finish_item(seq_item);
46              end
47
48          endtask
49
50      endclass
```

```
52      class FIFO_read_sequence extends uvm_sequence #(FIFO_sequence_item);
53          `uvm_object_utils(FIFO_read_sequence)
54
55          FIFO_sequence_item seq_item;
56
57          function new(string name = "FIFO_read_sequence");
58              super.new(name);
59          endfunction
60
61          task body();
62              seq_item = FIFO_sequence_item::type_id::create("seq_item");
63
64              repeat(10) begin
65                  start_item(seq_item);
66                  seq_item.rst_n= 1;
67                  seq_item.wr_en= 0;
68                  seq_item.rd_en= 1;
69                  finish_item(seq_item);
70              end
71
72          endtask
73      endclass
```

```
    class FIFO_rd_wr_sequence extends uvm_sequence #(FIFO_sequence_item);
        `uvm_object_utils(FIFO_rd_wr_sequence)

        FIFO_sequence_item seq_item;

        function new(string name = "FIFO_rd_wr_sequence");
            super.new(name);
        endfunction

        task body();
            seq_item = FIFO_sequence_item::type_id::create("seq_item");

            repeat(30000) begin
                start_item(seq_item);
                assert(seq_item.randomize());
                finish_item(seq_item);
            end
        endtask
    endclass
endpackage
```

```
package FIFO_agent_pkg;
import uvm_pkg::*;
import FIFO_config_pkg::*;
import FIFO_monitor_pkg::*;
import FIFO_driver_pkg::*;
import FIFO_sequence_item_pkg::*;
import FIFO_sequencer_pkg::*;
`include "uvm_macros.svh"

    class FIFO_agent extends uvm_agent;
        `uvm_component_utils(FIFO_agent);

        FIFO_monitor mon;
        FIFO_driver drv;
        FIFO_sequencer sqr;
        FIFO_config FIFO_cfg;
        uvm_analysis_port #(FIFO_sequence_item) agt_ap;

        function new(string name = "FIFO_agent", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            mon = FIFO_monitor::type_id::create("mon",this);
            drv = FIFO_driver::type_id::create("drv",this);
            sqr = FIFO_sequencer::type_id::create("sqr",this);
            agt_ap = new("agt_ap",this);

            if(!uvm_config_db #(FIFO_config)::get(this,"","CFG",FIFO_cfg))
                `uvm_fatal("build phase","Agent - Unable to get configuration object");

        endfunction
```

```
        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            mon = FIFO_monitor::type_id::create("mon",this);
            drv = FIFO_driver::type_id::create("drv",this);
            sqr = FIFO_sequencer::type_id::create("sqr",this);
            agt_ap = new("agt_ap",this);

            if(!uvm_config_db #(FIFO_config)::get(this,"","CFG",FIFO_cfg))
                `uvm_fatal("build phase","Agent - Unable to get configuration object");

        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            drv.fifo_vif = FIFO_cfg.fifo_vif;
            mon.fifo_vif = FIFO_cfg.fifo_vif;
            drv.seq_item_port.connect(sqr.seq_item_export);
            mon.mon_ap.connect(agt_ap);
        endfunction

    endclass

endpackage
```

```
package FIFO_sequence_item_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

    class FIFO_sequence_item extends uvm_sequence_item;
        `uvm_object_utils(FIFO_sequence_item)

        rand logic [15:0] data_in;
        rand logic rst_n, wr_en, rd_en;

        logic [15:0] data_out;
        logic wr_ack, overflow;
        logic full, empty, almostfull, almostempty, underflow;

        function new(string name = "FIFO_sequence_item");
            super.new(name);
        endfunction

        function string convert2string();
            return $sformatf("%s rst_n = 0b%0b, wr_en = 0b%0b\n
                              rd_en = 0b%0b, data_in = 0b%0b\n
                              data_out = 0b%0b, wr_ack = 0b%0b, overflow = 0b%b\n
                              full = 0b%0b, empty = 0b%0b, almostfull= 0b%0b\n
                              almostempty = 0b%0b, underflow = 0b%0b", super.convert2string(),
                              rst_n, wr_en, rd_en, data_in, data_out, wr_ack, overflow,
                              full, empty, almostfull, almostempty, underflow);
        endfunction
```

```
        function string convert2string_stimulus();
            return $sformatf("rst_n = 0b%0b, wr_en = 0b%0b\n
                              rd_en = 0b%0b, data_in = 0b%0b\n
                              data_out = 0b%0b, wr_ack = 0b%0b, overflow = 0b%b\n
                              full = 0b%0b, empty = 0b%0b, almostfull= 0b%0b,\n
                              almostempty = 0b%0b, underflow = 0b%0b",
                              rst_n, wr_en, rd_en, data_in, data_out, wr_ack, overflow,
                              full, empty, almostfull, almostempty, underflow);
        endfunction

        // constraint
        constraint rst_cst{
            rst_n dist{0 := 2, 1 := 98};
        }

        constraint wr_cst{
            wr_en dist{0 := 30, 1 := 70};
        }

        constraint rd_cst{
            rd_en dist{0 := 70, 1 := 60};
        }

    endclass

endpackage
```

```
package FIFO_sequencer_pkg;
import uvm_pkg::*;
import FIFO_sequence_item_pkg::*;
`include "uvm_macros.svh"

    class FIFO_sequencer extends uvm_sequencer #(FIFO_sequence_item);
        `uvm_component_utils(FIFO_sequencer)

        function new(string name = "FIFO_sequencer",uvm_component parent = null);
            super.new(name,parent);
        endfunction

    endclass

endpackage
```

FIFO_driver.sv

```
package FIFO_driver_pkg;
import uvm_pkg::*;
import FIFO_config_pkg::*;
import FIFO_sequence_item_pkg::*;
`include "uvm_macros.svh"

    class FIFO_driver extends uvm_driver #(FIFO_sequence_item);
        `uvm_component_utils(FIFO_driver);

        virtual FIFO_if fifo_vif;
        FIFO_sequence_item stim_seq_item;

        function new(string name = "FIFO_driver", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            stim_seq_item = FIFO_sequence_item::type_id::create("stim_seq_item",this);
            forever begin
                seq_item_port.get_next_item(stim_seq_item);
                fifo_vif.data_in     =   stim_seq_item.data_in;
                fifo_vif.wr_en       =   stim_seq_item.wr_en;
                fifo_vif.rd_en       =   stim_seq_item.rd_en;
                fifo_vif.rst_n       =   stim_seq_item.rst_n;
                @(negedge fifo_vif.clk);
                seq_item_port.item_done();
                `uvm_info("DRIVER",stim_seq_item.convert2string_stimulus(),UVM_HIGH);
            end

        endtask

    endclass

endpackage
```

```
package FIFO_monitor_pkg;
import uvm_pkg::*;
import FIFO_config_pkg::*;
import FIFO_sequence_item_pkg::*;
`include "uvm_macros.svh"

    class FIFO_monitor extends uvm_monitor;
        `uvm_component_utils(FIFO_monitor);

        virtual FIFO_if fifo_vif;
        FIFO_sequence_item rsp_seq_item;
        uvm_analysis_port #(FIFO_sequence_item) mon_ap;

        function new(string name = "FIFO_monitor", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            mon_ap = new("mon_ap",this);
        endfunction
```

```
    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            rsp_seq_item = FIFO_sequence_item::type_id::create("rsp_seq_item",this);
            @(negedge fifo_vif.clk);
            rsp_seq_item.data_in     =   fifo_vif.data_in;
            rsp_seq_item.wr_en       =   fifo_vif.wr_en;
            rsp_seq_item.rd_en       =   fifo_vif.rd_en;
            rsp_seq_item.rst_n       =   fifo_vif.rst_n;
            rsp_seq_item.full        =   fifo_vif.full;
            rsp_seq_item.empty       =   fifo_vif.empty;
            rsp_seq_item.almostfull  =   fifo_vif.almostfull;
            rsp_seq_item.almostempty =   fifo_vif.almostempty;
            rsp_seq_item.wr_ack      =   fifo_vif.wr_ack;
            rsp_seq_item.overflow    =   fifo_vif.overflow;
            rsp_seq_item.underflow   =   fifo_vif.underflow;
            rsp_seq_item.data_out    =   fifo_vif.data_out;
            mon_ap.write(rsp_seq_item); //broadcast
            `uvm_info("run_phase",rsp_seq_item.convert2string_stimulus(),UVM_HIGH);
        end
    endtask

endclass

endpackage
```

FIFO_scoreboard.sv

```
1   package FIFO_scoreboard_pkg;
2   import uvm_pkg::*;
3   import FIFO_sequence_item_pkg::*;
4   `include "uvm_macros.svh"
5
6       class FIFO_scoreboard extends uvm_scoreboard;
7           `uvm_component_utils(FIFO_scoreboard);
8
9           uvm_analysis_export #(FIFO_sequence_item) sb_export;
10          uvm_tlm_analysis_fifo #(FIFO_sequence_item) sb_fifo;
11          FIFO_sequence_item sb_seq_item;
12
13          // int correct_count = 0;
14          // int error_count = 0;
15
16          function new(string name = "FIFO_env", uvm_component parent = null);
17              super.new(name,parent);
18              sb_export = new("sb_export",this);
19              sb_fifo = new("sb_fifo",this);
20          endfunction
21
22          function void connect_phase(uvm_phase phase);
23              super.connect_phase(phase);
24              sb_export.connect(sb_fifo.analysis_export);
25          endfunction
26
27           task run_phase(uvm_phase phase);
28              super.run_phase(phase);
29              forever begin
30                  sb_fifo.get(sb_seq_item);
31              end
32           endtask
33
34      endclass
35
36  endpackage
37
```

```
package FIFO_coverage_pkg;
import uvm_pkg::*;
import FIFO_sequence_item_pkg::*;
`include "uvm_macros.svh"

    class FIFO_coverage extends uvm_component;
        `uvm_component_utils(FIFO_coverage);

        uvm_analysis_export #(FIFO_sequence_item) cv_export;
        uvm_tlm_analysis_fifo #(FIFO_sequence_item) cv_fifo;
        FIFO_sequence_item cv_seq_item;

        covergroup FIFO_cvg;

            wr_cvg: coverpoint cv_seq_item.wr_en;
            rd_cvg: coverpoint cv_seq_item.rd_en;
            wr_ack_cvg : coverpoint cv_seq_item.wr_ack;
            overflow_cvg : coverpoint cv_seq_item.overflow;
            full_cvg : coverpoint cv_seq_item.full;
            empty_cvg : coverpoint cv_seq_item.empty;
            almostfull_cvg : coverpoint cv_seq_item.almostfull;
            almostempt_cvg : coverpoint cv_seq_item.almostempty;
            underflow_cvg : coverpoint cv_seq_item.underflow;

            wr_ack_C: cross wr_cvg, rd_cvg , wr_ack_cvg;
            overflow_C: cross wr_cvg, rd_cvg , overflow_cvg;
            full_C: cross wr_cvg, rd_cvg , full_cvg;
            empty_C: cross wr_cvg, rd_cvg , empty_cvg;
            almostfull_C: cross wr_cvg, rd_cvg , almostfull_cvg;
            almostempty_C: cross wr_cvg, rd_cvg , almostempt_cvg;
            underflow_C: cross wr_cvg, rd_cvg , underflow_cvg;

        endgroup
```

```
        function new(string name = "FIFO_coverage", uvm_component parent = null);
            super.new(name,parent);
            FIFO_cvg = new();
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            cv_export = new("cv_export",this);
            cv_fifo = new("cv_fifo",this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            cv_export.connect(cv_fifo.analysis_export);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                cv_fifo.get(cv_seq_item);
                FIFO_cvg.sample();
            end
        endtask

    endclass
```

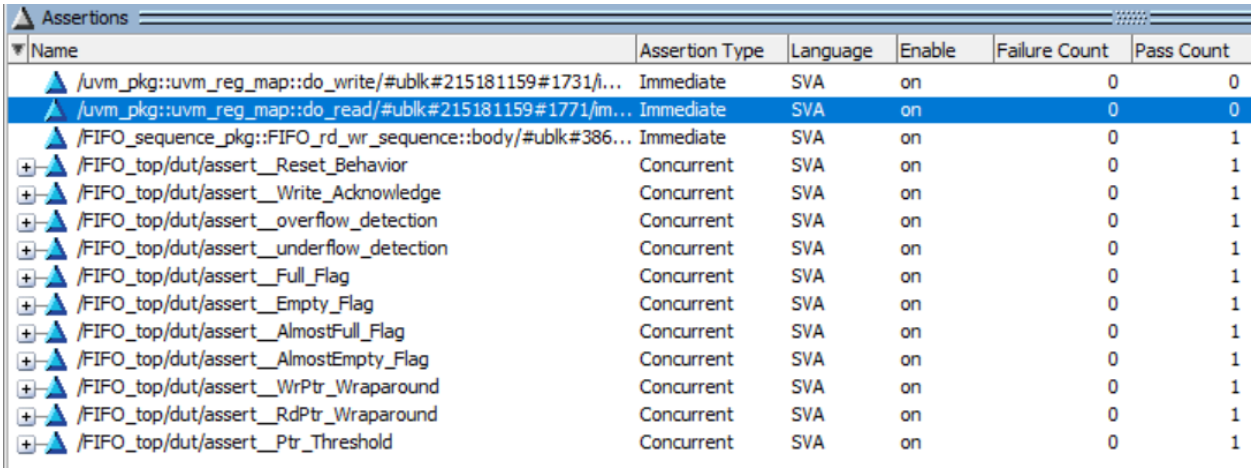
src_files.list

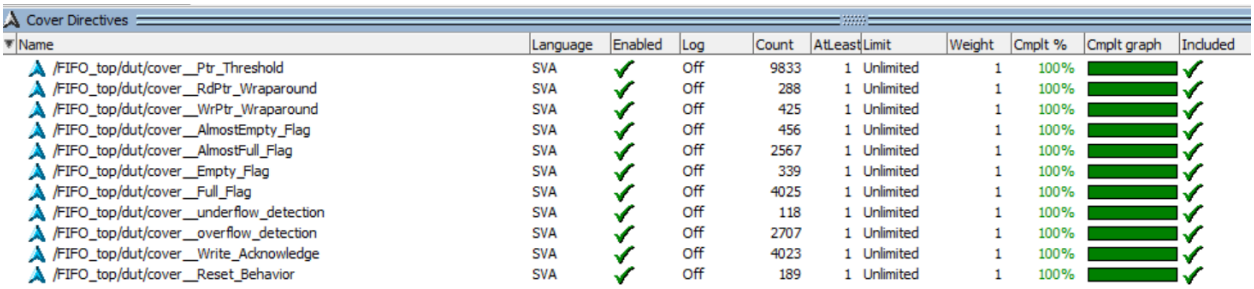
```
FIFO_sequenceitem.sv
FIFO_sequence.sv
FIFO_config.sv
FIFO_if.sv
FIFO_driver.sv
FIFO_monitor.sv
FIFO_coverage.sv
FIFO_scoreboard.sv
FIFO_sequencer.sv
FIFO_agent.sv
FIFO_env.sv
FIFO_test.sv
FIFO.sv
FIFO_top.sv
```

run.do

```
vlib work

vlog -sv +define+SIM +cover=bcesf -f src_files.list

vsim work.FIFO_top -voptargs=+acc -classdebug -uvmcontrol=all +UVM_TESTNAME=FIFO_test -coverage

add wave /FIFO_top/dut/*
add wave /FIFO_top/dut/assert__Reset_Behavior \
         /FIFO_top/dut/assert__Write_Acknowledge \
         /FIFO_top/dut/assert__overflow_detection \
         /FIFO_top/dut/assert__underflow_detection \
         /FIFO_top/dut/assert__Full_Flag \
         /FIFO_top/dut/assert__Empty_Flag \
         /FIFO_top/dut/assert__AlmostFull_Flag \
         /FIFO_top/dut/assert__AlmostEmpty_Flag \
         /FIFO_top/dut/assert__WrPtr_Wraparound \
         /FIFO_top/dut/assert__RdPtr_Wraparound \
         /FIFO_top/dut/assert__Ptr_Threshold
run -all

coverage save FIFO_coverage.ucdb

vcover report FIFO_coverage.ucdb -details -annotate -all -output coverage_rpt.txt
```

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	When the reset is asserted, the output ant pointer value should be low	Directed at the start of the simulation	-	Assertions + Scoreboard compares expected vs actual stored data.
FIFO_2	When wr_en is asserted and FIFO is not full, data_in shall be written into memory and wr_ack shall assert.	Constrained random writes.	Cover cross coverage between wr_en, rd_en wr_ack, and write success cases.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_3	When wr_en is asserted while FIFO is full, no write shall occur and overflow shall assert.	Directed + random (force full condition then write).	Cover cross coverage between wr_en, rd_en, overflow	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_4	When rd_en is asserted and FIFO is not empty, data shall be read correctly from memory.	Constrained random reads.	Cover cross coverage between wr_en, rd_en full	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_5	When rd_en is asserted while FIFO is empty, no read shall occur and underflow shall assert.	Directed + random (force empty condition then read).	Cover cross coverage between wr_en, rd_en, underflow	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_6	FIFO shall assert full when count equals FIFO depth.	Random fill sequences.	Cover full condition.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_7	Simultaneous read and write operations shall update pointers and count correctly.	Constrained random with wr_en & rd_en both high.	Cross coverage of wr_en and rd_en.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_8	Write pointer shall wrap around correctly at FIFO depth.	Directed wrap-around sequences.	Cover pointer boundary conditions.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_9	Read pointer shall wrap around correctly at FIFO depth.	Directed wrap-around sequences.	Cover pointer boundary conditions.	Assertions + Scoreboard compares expected vs actual stored data..
FIFO_10	FIFO count shall never exceed depth or go below zero.	Directed wrap-around sequences.	Cover pointer boundary conditions.	Assertions + Scoreboard compares expected vs actual stored data..





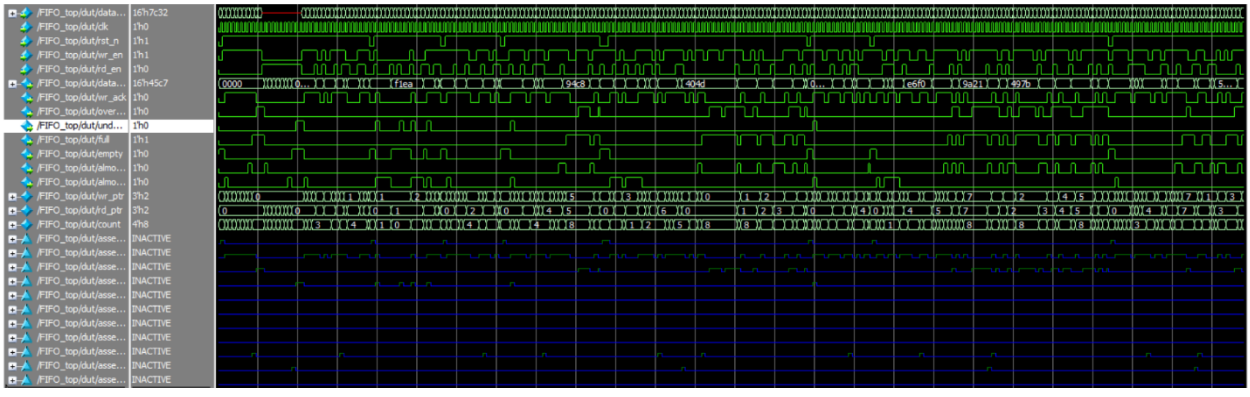
FIFO.sv

```
24 always @(posedge clk or negedge rst_n) begin
26 wr_ptr <= 0;
27 wr_ack <= 0;
28 overflow <= 0;
31 mem[wr_ptr] <= data_in;
32 wr_ack <= 1;
33 wr_ptr <= wr_ptr + 1;
36 wr_ack <= 0;
38 overflow <= 1;
40 overflow <= 0;
44 always @(posedge clk or negedge rst_n) begin
46 rd_ptr <= 0;
47 data_out <= 0;
48 underflow <= 0;
51 data_out <= mem[rd_ptr];
52 rd_ptr <= rd_ptr + 1;
53 underflow <= 0;
56 underflow <= 1;
58 underflow <= 0;
62 always @(posedge clk or negedge rst_n) begin
64 count <= 0;
68 count <= count + 1;
70 count <= count - 1;
73 count <= count + 1;
75 count <= count - 1;
80 assign full = (count == FIFO_DEPTH)? 1 : 0;
81 assign empty = (count == 0)? 1 : 0;
83 assign almostfull = (count == FIFO_DEPTH-1)? 1 : 0;
84 assign almostempty = (count == 1)? 1 : 0;
```

FIFO.sv

```
25 if (!rst_n) begin
30 else if (wr_en && count < FIFO_DEPTH) begin
35 else begin
37 if (full & wr_en)
39 else
45 if (!rst_n) begin
50 else if (rd_en && count != 0) begin
54 end  else begin
55 if (rd_en && empty) // this synchronized
57 else
63 if (!rst_n) begin
66 else begin
67 if       ( ({wr_en, rd_en} == 2'b10) && !full)
69 else if ( ({wr_en, rd_en} == 2'b01) && !empty)
71 else if({wr_en, rd_en} == 2'b11) begin // this condition was added
72 if(empty)
74 else if(full)
80 assign full = (count == FIFO_DEPTH)? 1 : 0;
81 assign empty = (count == 0)? 1 : 0;
83 assign almostfull = (count == FIFO_DEPTH-1)? 1 : 0;
84 assign almostempty = (count == 1)? 1 : 0;
```

```
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    8
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [run_phase]     4
# ** Note: $finish    : C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 100220 ns  Iteration: 54  Instance: /FIFO_top
# 1
```



Assertion Name	SVA Code	Description
assert__Reset_Behavior	@(posedge clk) (!rst_n) |=> (count == 0 && wr_ptr == 0 && rd_ptr == 0);	After reset is asserted (rst_n = 0), count, wr_ptr, and rd_ptr must all be 0 on the next clock cycle.
assert__Write_Acknowledge	@(posedge clk) disable iff (!rst_n) (wr_en && !full) |=> (wr_ack == 1);	When a write is enabled and the FIFO is not full, wr_ack must be high on the next clock cycle.
assert__overflow_detection	@(posedge clk) disable iff (!rst_n) (wr_en && full) |=> (overflow == 1);	When a write is attempted on a full FIFO, the overflow flag must be asserted on the next clock cycle.
assert__underflow_detection	@(posedge clk) disable iff (!rst_n) (rd_en && empty) |=> (underflow == 1);	When a read is attempted on an empty FIFO, the underflow flag must be asserted on the next clock cycle.
assert__Full_Flag	@(posedge clk) disable iff (!rst_n) (count == FIFO_DEPTH) |-> (full == 1);	When count equals FIFO_DEPTH, the full flag must be immediately (same cycle) asserted.
assert__Empty_Flag	@(posedge clk) disable iff (!rst_n) (count == 0) |-> (empty == 1);	When count equals 0, the empty flag must be immediately (same cycle) asserted.
assert__AlmostFull_Flag	@(posedge clk) disable iff (!rst_n) (count == FIFO_DEPTH - 1) |-> (almostfull == 1);	When count equals FIFO_DEPTH-1, the almostfull flag must be immediately asserted.

Assertion Name	SVA Code	Description
assert__AlmostEmpty_Flag	@(posedge clk) disable iff (!rst_n) (count == 1) |-> (almostempty == 1);	When count equals 1, the almostempty flag must be immediately asserted.
assert__WrPtr_Wraparound	@(posedge clk) disable iff (!rst_n) (wr_ptr == FIFO_DEPTH-1 && wr_en && !full) |=> (wr_ptr == 0);	When the write pointer reaches FIFO_DEPTH-1 and a valid write occurs, it must wrap around to 0 on the next cycle.
assert__RdPtr_Wraparound	@(posedge clk) disable iff (!rst_n) (rd_ptr == FIFO_DEPTH-1 && rd_en && !empty) |=> (rd_ptr == 0);	When the read pointer reaches FIFO_DEPTH-1 and a valid read occurs, it must wrap around to 0 on the next cycle.
assert__Ptr_Threshold	@(posedge clk) disable iff (!rst_n) (wr_ptr < FIFO_DEPTH) && (rd_ptr < FIFO_DEPTH) && (count <= FIFO_DEPTH);	At all times, both pointers must remain within [0, FIFO_DEPTH-1] and count must never exceed FIFO_DEPTH.